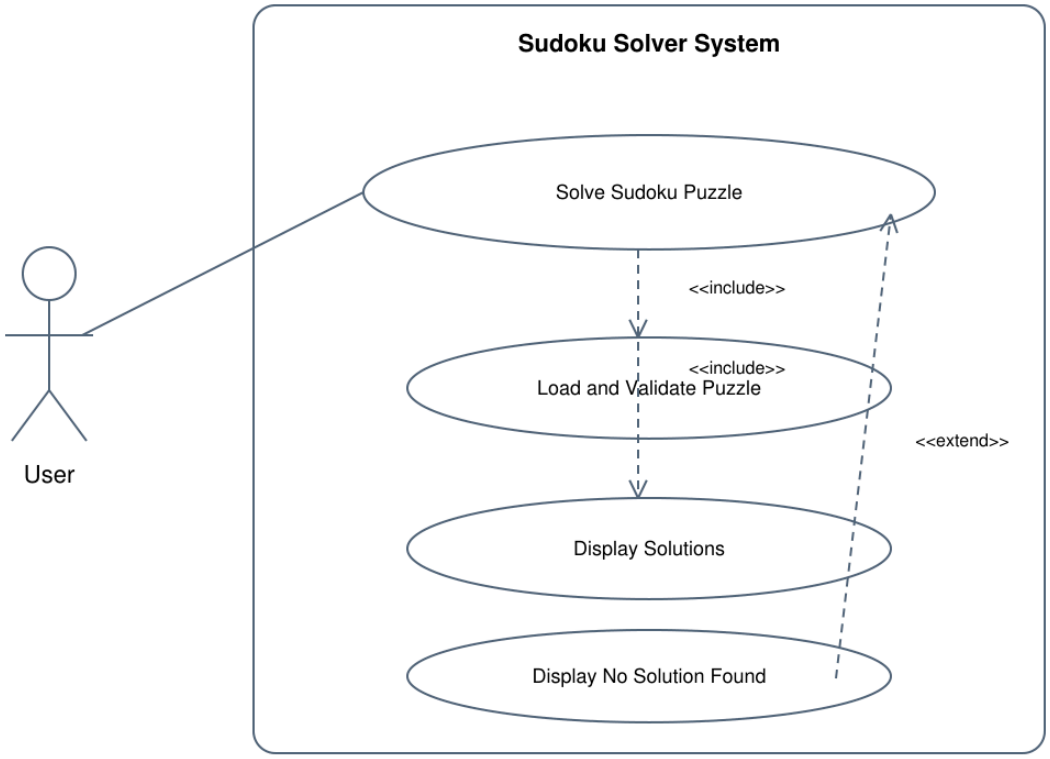
Sudoku Solver - UML Use Case Diagram

EECS 348 Assignment 4 | Actor: User | System: Sudoku Solver



Main use case: Solve Sudoku Puzzle

Precondition: The five puzzle files are available in the working directory.

Postcondition: Every valid solution is displayed, or a no-solution message is shown.

Brief description

The user runs the Sudoku application. The application loads each of the five puzzle files, displays its starting
grid, searches recursively for all valid solutions, and displays the results.

Primary actor

User (student or grader running the program).

Trigger

User launches the compiled executable in the folder containing the puzzle files.

Preconditions

The executable is available and the current working directory contains puzzle1.txt through puzzle5.txt.

Main success scenario

1. User launches Assignment4.exe.
2. System selects the next puzzle file.
3. System reads the 81 entries and validates the starting clues.
4. System prints the file name and starting grid.
5. System selects an empty square and determines legal candidates.
6. System tentatively assigns a candidate and recursively searches.
7.Whenever a complete valid grid is found, the system prints it.
8. System backtracks to find any additional solutions.
9. System prints the total number of solutions and proceeds to the next file.
10. Program exits after all five files.

Alternate and exception flows

A1: If a file cannot be opened, is malformed, or has conflicting clues, the system prints an error and continues to the next file.

A2: If an empty square has no legal candidates, the recursive branch returns and tries another candidate.

A3: If no complete solution exists, the system prints "No solution found." A4: If more than one solution exists, the system prints all solutions, not just the first.

Postconditions

Each valid input puzzle has its original grid and all found solutions displayed, or an explicit no-solution result.

Implementation details

SudokuSolver owns a 9-by-9 grid. load() validates input; allowed() checks row, column, and 3-by-3 block constraints; chooseCell() selects the blank with fewest candidates; backtrack() recursively enumerates all solutions; print() formats the grid; solve() reports results.